BottleNeck, essais complémentaires

Projet 13, partie 1. Complément du notebook amélioré (`Lippens_Charles_1_projet_ameliore_092026.ipynb` dans le dossier de dépôt). Il se range dans le même dossier et lit les mêmes données (`Data+Bottleneck`).

Auteur	Charles Lippens
Objet	Exécuter ce que le notebook amélioré écarte ou cite sans le calculer : Local Outlier Factor (essai A, axe 3), importances du modèle face aux valeurs de Shapley (essai B, axe 4), chiffres du réassort (essai C, besoin 2), stabilité du modèle et référence naïve (essai D, axe 4)
Dates	Essais A et B le 17 septembre 2026 ; essais C et D le 25 septembre 2026, avec une nouvelle exécution complète le même jour (mêmes résultats pour A et B) ; textes revus le 28 septembre et le 14 octobre 2026, code et sorties inchangés
Origine	A et B : demande du 17 septembre, tirée de l'audit des attendus du même jour. C et D : suites de l'audit de l'énoncé du 25 septembre (décision de réassort à outiller, stabilité du modèle à mesurer)
Outil	Claude (Anthropic), le 17 et le 25 septembre
Journal	Documentation de la démarche, annexe A : sections A.3 et A.4 pour les essais A, B et D, section A.5 pour l'essai C
Essais C et D	Chiffres recalculés par l'assistant dans un script séparé ; relus et relancés par moi le 7 octobre 2026 : mêmes chiffres

La préparation (chargement, nettoyage, jointure, score z, Isolation Forest) reprend le code du notebook amélioré, avec la même graine : les essais portent sur les mêmes 714 produits.

[1]

```
import warnings, platform, random
warnings.filterwarnings('ignore')
import numpy as np, pandas as pd
from pathlib import Path

SEED = 42
random.seed(SEED); np.random.seed(SEED)
pd.set_option('display.max_columns', None)
print('Python', platform.python_version())

CANDIDATS = [Path('Data+Bottleneck'), Path('../Data+Bottleneck'),
             Path('P6_donnees/Data+Bottleneck'), Path('../P6_donnees/Data+Bottleneck')]
DATA_DIR = next((p for p in CANDIDATS if (p/'erp.xlsx').exists()), None)
assert DATA_DIR is not None, "Placez erp/web/liaison.xlsx dans un dossier 'Data+Bottleneck' à côté du notebook."
erp     = pd.read_excel(DATA_DIR/'erp.xlsx')
web     = pd.read_excel(DATA_DIR/'web.xlsx')
liaison = pd.read_excel(DATA_DIR/'liaison.xlsx')
print('erp', erp.shape, '| web', web.shape, '| liaison', liaison.shape)
```

```
Python 3.11.15
erp (825, 6) | web (1513, 29) | liaison (825, 2)
```

Préparation, reprise à l'identique du notebook amélioré (sections 2.1, 2.3, 4.1 et 4.2)

[2]

```
web_prod = web[web['post_type']=='product'].copy()
lia = liaison.dropna(subset=['id_web']).copy(); lia['id_web'] = lia['id_web'].astype(str)
wpc = web_prod.dropna(subset=['sku']).drop_duplicates(subset=['sku']).copy(); wpc['sku'] =
wpc['sku'].astype(str)

df = (erp.merge(lia, on='product_id', how='inner')
         .merge(wpc, left_on='id_web', right_on='sku', how='inner'))
df['ca_par_article'] = df['price'] * df['total_sales']
df['marge_unitaire'] = df['price'] - df['purchase_price']
df['taux_marge']     = np.where(df['price']>0, df['marge_unitaire']/df['price']*100, np.nan)
print(f"Périmètre analysable : {len(df)} produits")

# score z sur le seul prix (méthode du projet 6)
mu, sd = df['price'].mean(), df['price'].std()
df['z'] = (df['price']-mu)/sd
df['z_anom'] = df['z'].abs() > 3

# Isolation Forest, réglages du notebook amélioré
from sklearn.ensemble import IsolationForest
feats = ['price','purchase_price','stock_quantity','total_sales','taux_marge']
X = df[feats].fillna(df[feats].median())
iso = IsolationForest(contamination=0.05, n_estimators=200, random_state=SEED)
df['if_anom'] = iso.fit_predict(X) == -1
print(f"Isolation Forest : {int(df['if_anom'].sum())} anomalies | Z-score : {int(df['z_anom'].sum())} |
communes : {int((df['if_anom'] & df['z_anom']).sum())}")
```

```
Périmètre analysable : 714 produits
Isolation Forest : 36 anomalies | Z-score : 13 | communes : 10
```

Essai A, Local Outlier Factor face à Isolation Forest (axe 3)

Les cinq variables sont les mêmes, standardisées car Local Outlier Factor repose sur des distances, et le taux reste de 5 %, ce qui fixe le nombre d'anomalies à 36. Je fais varier son réglage principal, le nombre de voisins. Je mesure le recouvrement avec Isolation Forest et avec le score z, puis la stabilité d'un réglage au suivant : l'indice de Jaccard entre les deux listes d'anomalies. Pour comparer, je mesure la stabilité d'Isolation Forest en changeant sa graine.

[3]

```
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler

Xs = StandardScaler().fit_transform(X)
lignes, ensembles = [], {}
for k in [5, 10, 20, 35, 50]:
    lof = LocalOutlierFactor(n_neighbors=k, contamination=0.05)
    anom = lof.fit_predict(Xs) == -1
    ensembles[k] = set(df.index[anom])
    lignes.append({'n_neighbors': k,
                   'anomalies LOF': int(anom.sum()),
                   'communes avec Isolation Forest': int((anom & df['if_anom']).sum()),
                   'communes avec le score z': int((anom & df['z_anom']).sum())})
res = pd.DataFrame(lignes).set_index('n_neighbors')

# stabilité : indice de Jaccard entre deux réglages successifs
ks = list(ensembles)
jac = [np.nan] + [len(ensembles[a] & ensembles[b]) / len(ensembles[a] | ensembles[b]) for a, b in
zip(ks, ks[1:])]
res['Jaccard avec le réglage précédent'] = np.round(jac, 2)
display(res)

# Isolation Forest, même mesure de stabilité en faisant varier la graine (sans changer le réglage)
ens_if = [set(df.index[IsolationForest(contamination=0.05, n_estimators=200,
random_state=s).fit_predict(X) == -1]) for s in [42, 1, 7, 2026]]
jac_if = [len(ens_if[0] & e) / len(ens_if[0] | e) for e in ens_if[1:]]
print('Isolation Forest, Jaccard entre la graine 42 et trois autres graines :', np.round(jac_if, 2))
print("Référence affichée à 12,65 EUR (achat 77,48 EUR) repérée par Local Outlier Factor selon le
réglage :",
      {k: ('oui' if 201 in e else 'non') for k, e in ensembles.items()})
```

n_neighbors	anomalies LOF	communes avec Isolation Forest	communes avec le score z	Jaccard avec le réglage précédent
5	36	13	6	NaN
10	36	13	7	0.67
20	36	11	5	0.76
35	36	22	5	0.41
50	36	22	5	1.00

```
Isolation Forest, Jaccard entre la graine 42 et trois autres graines : [0.8  0.8  0.85]
Référence affichée à 12,65 EUR (achat 77,48 EUR) repérée par Local Outlier Factor selon le réglage : {5:
'oui', 10: 'oui', 20: 'oui', 35: 'oui', 50: 'oui'}
```

Lecture. Selon le nombre de voisins, Local Outlier Factor ne partage que 11 à 22 de ses 36 anomalies avec Isolation Forest. Sa liste change aussi d'un réglage à l'autre : l'indice de Jaccard va de 0,41 (entre 20 et 35 voisins) à 1. Isolation Forest varie moins quand on change sa graine (0,80 à 0,85). Les deux méthodes repèrent la référence affichée à 12,65 euros. Je garde Isolation Forest. Réserve : les deux mesures de stabilité ne portent pas sur la même chose, le réglage pour l'un, la graine pour l'autre.

Essai B, importances globales face aux valeurs de Shapley (axe 4)

Même modèle que le notebook amélioré : gradient boosting, 300 arbres de profondeur 3, taux d'apprentissage 0,05, même partition 75 % / 25 %, même graine. Je compare trois lectures des facteurs de vente : l'importance du modèle (réduction d'impureté), l'importance par permutation sur le jeu de test, et la moyenne des valeurs absolues de Shapley sur le jeu de test. Enfin, un exemple sur une référence montre ce que seules les valeurs de Shapley apportent : la part de chaque variable dans une prédiction.

[4]

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.inspection import permutation_importance
import shap

mfeats =
['price','purchase_price','taux_marge','stock_quantity','average_rating','rating_count','onsale_web']
M = df[mfeats].fillna(0); y = df['total_sales']
Xtr, Xte, ytr, yte = train_test_split(M, y, test_size=0.25, random_state=SEED)
gb = GradientBoostingRegressor(n_estimators=300, max_depth=3, learning_rate=0.05,
random_state=SEED).fit(Xtr, ytr)
print(f"R2 (test) = {r2_score(yte, gb.predict(Xte)):.3f}  |  MAE = {mean_absolute_error(yte,
gb.predict(Xte)):.2f} unités")

perm = permutation_importance(gb, Xte, yte, n_repeats=20, random_state=SEED)
sv = shap.TreeExplainer(gb).shap_values(Xte)
comp = pd.DataFrame({
    'importance du modèle (impureté)': gb.feature_importances_,
    'importance par permutation (test)': perm.importances_mean,
    'moyenne des |valeurs de Shapley| (test)': np.abs(sv).mean(axis=0),
}, index=mfeats)
comp = comp.div(comp.sum(axis=0), axis=1).round(3)   # chaque colonne ramenée à des parts (total 1) pour
comparer les classements
comp['rang modèle'] = comp.iloc[:, 0].rank(ascending=False).astype(int)
comp['rang permutation'] = comp.iloc[:, 1].rank(ascending=False).astype(int)
comp['rang Shapley'] = comp.iloc[:, 2].rank(ascending=False).astype(int)
display(comp.sort_values('rang Shapley'))
```

```
R2 (test) = 0.724  |  MAE = 1.54 unités
```

	importance du modèle (impureté)	importance par permutation (test)	moyenne des |valeurs de Shapley| (test)	rang modèle	rang permutation	rang Shapley
stock_quantity	0.683	0.628	0.555	1	1	1
purchase_price	0.164	0.221	0.261	2	2	2
taux_marge	0.090	0.109	0.106	3	3	3
price	0.063	0.043	0.077	4	4	4
average_rating	0.000	0.000	0.000	6	6	6
rating_count	0.000	0.000	0.000	6	6	6
onsale_web	0.000	0.000	0.000	6	6	6

[5]

```
# Un cas individuel : la référence du jeu de test dont la prédiction est la plus élevée
i = int(np.argmax(gb.predict(Xte)))
base = float(np.ravel(shap.TreeExplainer(gb).expected_value)[0])
contrib = pd.Series(sv[i], index=mfeats).sort_values(key=np.abs, ascending=False)
print(f"Ligne {Xte.index[i]} du jeu de test (produit {df.loc[Xte.index[i], 'product_id']}) : ventes
réelles {yte.iloc[i]:.0f}, "
      f"prédites {gb.predict(Xte)[i]:.1f}, valeur de base {base:.1f}")
print('Contribution de chaque variable à cette prédiction (unités de ventes) :')
display(contrib.round(2).to_frame('contribution'))
```

```
Ligne 135 du jeu de test (produit 4220) : ventes réelles 18, prédites 17.0, valeur de base 8.1
Contribution de chaque variable à cette prédiction (unités de ventes) :
```

	contribution
stock_quantity	5.75
purchase_price	2.32
taux_marge	1.41
price	-0.58
average_rating	0.00
rating_count	0.00
onsale_web	0.00

Lecture. Les trois mesures donnent le même classement : stock, prix d'achat, taux de marge, puis prix. Les colonnes d'avis et `onsale_web` ne comptent pas. Si je retiens les valeurs de Shapley, c'est donc pour expliquer une référence, pas pour le classement global. Pour le produit 4220 (ligne 135 de la table, dans le jeu de test), le modèle prédit 17 ventes contre 8,1 en moyenne ; le stock explique 5,75 de cet écart de 8,9.

Essai C, les chiffres du réassort (besoin 2, savoir où agir)

Le notebook amélioré cite la marge et les ruptures à l'étape 6 sans les calculer : les voici, sur la même table de 714 produits. J'ajoute ensuite ce qui manquait pour décider du réassort : quelles références clés sont en rupture, et lesquelles ont trop de stock au rythme des ventes d'octobre. La couverture est calculée sans les deux lignes qui ne respectent pas le contrat de qualité (un stock négatif, un prix sous le prix d'achat).

[6]

```python
def fr(x, d=0):
    """Nombre au format français : espace pour les milliers, virgule décimale."""
    return f"{x:,.{d}f}".replace(',', ' ').replace('.', ',')

# marge et ruptures sur les 714 produits
rupture = df['stock_quantity'] <= 0
print(f"Marge moyenne : {fr(df['taux_marge'].mean(), 1)} % | médiane : {fr(df['taux_marge'].median(),
1)} %")
print(f"Produits en rupture (stock nul ou négatif) : {int(rupture.sum())}")

# références qui font 80 % du chiffre d'affaires, même calcul qu'à l'étape 6
ca_total = df['ca_par_article'].sum()
s = df[df['ca_par_article'] > 0].sort_values('ca_par_article', ascending=False).copy()
s['cum'] = s['ca_par_article'].cumsum() / ca_total * 100
cles = s.iloc[:int((s['cum'] <= 80).sum()) + 1]
nul, neg = cles[cles['stock_quantity'] == 0], cles[cles['stock_quantity'] < 0]
print(f"Références clés : {len(cles)} | stock nul : {len(nul)}, pour {fr(nul['ca_par_article'].sum())}
EUR de ventes en octobre"
      f" | stock négatif : {len(neg)}")
display(nul.sort_values('ca_par_article', ascending=False)
           [['product_id', 'post_title', 'total_sales', 'ca_par_article', 'stock_status']].head(5))

# couverture de stock au rythme des ventes d'octobre, sans les lignes hors contrat
ok = (df['price'] > 0) & (df['stock_quantity'] >= 0) & (df['purchase_price'] <= df['price'])
c = df[ok].copy()
c['valeur_achat'] = c['stock_quantity'] * c['purchase_price']
c['couverture_mois'] = c['stock_quantity'] / c['total_sales'].where(c['total_sales'] > 0)
long = c[c['couverture_mois'] > 12]
part = 100 * long['valeur_achat'].sum() / c['valeur_achat'].sum()
print(f"Plus d'un an de stock : {len(long)} références, dont
{int(long['post_title'].str.startswith('Champagne').sum())} champagnes ; "
      f"{fr(long['valeur_achat'].sum())} EUR au prix d'achat, soit {fr(part)} % de la valeur du stock")
display(long.sort_values('valeur_achat', ascending=False)
            [['product_id', 'post_title', 'stock_quantity', 'total_sales', 'couverture_mois',
'valeur_achat']].head(5).round(1))

# statut de stock contraire à la quantité
contraire = df[((df['stock_status'] == 'outofstock') & (df['stock_quantity'] > 0)) |
               ((df['stock_status'] == 'instock') & (df['stock_quantity'] <= 0))]
print(f"Statut contraire à la quantité : {len(contraire)} fiche(s) parmi les 714")
display(contraire[['product_id', 'stock_quantity', 'stock_status']])
```

```
Marge moyenne : 47,4 % | médiane : 48,3 %
Produits en rupture (stock nul ou négatif) : 45
Références clés : 435 | stock nul : 14, pour 5 434 EUR de ventes en octobre | stock négatif : 1
```

	product_id	post_title	total_sales	ca_par_article	stock_status
199	4352	Champagne Egly-Ouriet Grand Cru Millésimé 2008	11.0	2475.0	outofstock
479	5487	Decelle-Villa Nuits-Saint-Georges Rouge 2016	9.0	391.5	outofstock
667	6299	Lucien Boillot Gevrey Chambertin 1er Cru Les C…	4.0	312.0	outofstock
321	4726	François Baur Pinot Noir Schlittweg 2017	22.0	279.4	outofstock
516	5617	Albert Boxler Pinot Gris Réserve 2016	9.0	250.2	outofstock

```
Plus d'un an de stock : 24 références, dont 20 champagnes ; 95 012 EUR au prix d'achat, soit 35 % de la
valeur du stock
```

	product_id	post_title	stock_quantity	total_sales	couverture_mois	valeur_acha
587	5892	Coteaux Champenois Egly-Ouriet Ambonnay Rouge …	98	6.0	16.3	11373.
642	6126	Champagne Gosset Célébris Vintage 2007	138	5.0	27.6	11085.
437	5025	Champagne Agrapart & Fils L'Avizoise Extra…	136	6.0	22.7	9329.
198	4350	Champagne Egly-Ouriet Grand Cru Extra Brut V.P.	145	7.0	20.7	6858.
205	4359	Champagne Larmandier-Bernier Grand Cru Les Che…	112	7.0	16.0	5816.

```
Statut contraire à la quantité : 2 fiche(s) parmi les 714
```

	product_id	stock_quantity	stock_status
4	4039	3	outofstock
368	4885	0	instock

Lecture. La marge moyenne est de 47,4 % (médiane 48,3 %) et 45 produits sont en rupture : ce sont les valeurs citées à l'étape 6 du notebook amélioré. Parmi les 435 références qui font 80 % du chiffre d'affaires, 14 ont un stock nul ; elles ont fait 5 434 euros de ventes en octobre, dont 2 475 pour un seul champagne. Une quinzième a un stock négatif, déjà signalée par le contrat de qualité. L'une des 14, la référence 4885, est même marquée « en stock » dans l'ERP alors que son stock est nul. À l'inverse, 24 références, dont 20 champagnes, ont plus d'un an de stock au rythme d'octobre. Ce stock vaut environ 95 000 euros au prix d'achat, soit 35 % de la valeur du stock des 712 produits retenus.

Deux réserves : il n'y a qu'un mois de ventes, et les champagnes se vendent surtout en fin d'année. La couverture est donc à recalculer chaque mois. Décision : ces deux listes, réassort urgent et stock à écouler, deviennent la fonctionnalité F8 du cahier des charges.

Essai D, stabilité du modèle et référence naïve (axe 4)

Le cahier des charges laissait deux questions ouvertes. Le modèle fait-il mieux qu'une prédiction naïve ? Son pouvoir explicatif de 0,72 tient-il si l'on change la partition des données ? Même modèle et même graine que l'essai B.

[7]

```
from sklearn.model_selection import KFold, cross_validate

# 1. Référence naïve : prédire pour chaque référence la moyenne des ventes du jeu d'apprentissage
mae_modele = mean_absolute_error(yte, gb.predict(Xte))
mae_naif = mean_absolute_error(yte, np.full(len(yte), ytr.mean()))
print(f"Même partition que le notebook : erreur moyenne de {fr(mae_modele, 2)} unité pour le modèle, "
      f"{fr(mae_naif, 2)} unités pour la référence naïve")

# 2. Validation croisée à cinq plis, même modèle, même graine
plis = KFold(n_splits=5, shuffle=True, random_state=SEED)
modele = GradientBoostingRegressor(n_estimators=300, max_depth=3, learning_rate=0.05, random_state=SEED)
cv = cross_validate(modele, M, y, cv=plis, scoring=('r2', 'neg_mean_absolute_error'))
r2, mae = cv['test_r2'], -cv['test_neg_mean_absolute_error']
naif = [mean_absolute_error(y.iloc[te], np.full(len(te), y.iloc[tr].mean())) for tr, te in
plis.split(M)]
print('Pouvoir explicatif (R²) par pli :', ' ; '.join(fr(v, 2) for v in r2))
print(f"Moyenne {fr(r2.mean(), 2)}, de {fr(r2.min(), 2)} à {fr(r2.max(), 2)} | erreur moyenne
{fr(mae.mean(), 2)} unité, "
      f"référence naïve {fr(np.mean(naif), 2)} unités")

# 3. Classement des quatre premiers facteurs sur chaque pli (importance du modèle)
rangs = []
for tr, te in plis.split(M):
    m = GradientBoostingRegressor(n_estimators=300, max_depth=3, learning_rate=0.05,
random_state=SEED).fit(M.iloc[tr], y.iloc[tr])
    rangs.append(pd.Series(m.feature_importances_,
index=mfeats).sort_values(ascending=False).index[:4].tolist())
display(pd.DataFrame(rangs, index=[f'pli {i}' for i in range(1, 6)], columns=['1er', '2e', '3e', '4e']))
```

```
Même partition que le notebook : erreur moyenne de 1,54 unité pour le modèle, 3,46 unités pour la référence
naïve
```

```
Pouvoir explicatif (R²) par pli : 0,76 ; 0,68 ; 0,53 ; 0,73 ; 0,64
Moyenne 0,67, de 0,53 à 0,76 | erreur moyenne 1,51 unité, référence naïve 3,26 unités
```

	1er	2e	3e	4e
pli 1	stock_quantity	purchase_price	taux_marge	price
pli 2	stock_quantity	taux_marge	purchase_price	price
pli 3	stock_quantity	taux_marge	purchase_price	price
pli 4	stock_quantity	purchase_price	taux_marge	price
pli 5	stock_quantity	taux_marge	purchase_price	price

Lecture. Le modèle fait deux fois moins d'erreur que la prédiction naïve : 1,54 unité contre 3,46 sur la partition du notebook. Son pouvoir explicatif dépend de la partition : 0,67 en moyenne en validation croisée, de 0,53 à 0,76 selon le pli, pour 0,72 sur la partition du notebook. La cible du cahier des charges (0,60) est atteinte en moyenne mais pas sur chaque pli. Le classement des facteurs, lui, est stable : le stock est premier sur les cinq plis, le prix d'achat et le taux de marge échangent parfois leur place, le prix reste quatrième.

Décision : je présente le 0,72 avec sa fourchette, et le modèle sert à expliquer les ventes et non à les prévoir.

Versions

[8]

```
import sklearn, shap, matplotlib
for m in [np, pd, sklearn, shap, matplotlib]:
    print(f"{m.__name__:12s} {m.__version__}")
print('SEED =', SEED)
```

```
numpy        2.4.4
pandas       3.0.2
sklearn      1.8.0
shap         0.51.0
matplotlib   3.10.9
SEED = 42
```